

Uninformed Search: BFS, DFS, and Uniform-Cost Search

One graph-search skeleton, three frontier policies, different guarantees

Learning objectives

By the end of this lesson, you should be able to:

- 01 ——— Trace BFS, DFS, and UCS with a frontier and explored set.
- 02 ——— Define branching factor b , solution depth d , maximum depth m , and path cost g .
- 03 ——— Compare completeness, optimality, time, and space guarantees.
- 04 ——— Explain why graph search requires immutable state keys.
- 05 ——— Sketch why BFS is optimal for uniform costs and UCS for positive costs.
- 06 ——— Implement all three strategies behind one frontier interface.

Frontier ordering defines the search strategy

BFS, DFS, and UCS share the same expansion loop; only the removal priority changes.

- BFS removes the shallowest node.
- DFS removes the most recently added node.
- UCS removes the node with minimum accumulated path cost.

Policy

Queue, stack, and minimum-g priority produce different search behavior.

Search analysis uses four recurring quantities

**b = branching factor; d = shallowest solution depth; m = maximum depth;
 $g(n)$ = path cost**

- b bounds the number of successors generated per expansion.
- d counts edges to a shallowest goal.
- m bounds search depth when the space is finite.
- $g(n)$ sums step costs along node n 's path.

ENGINEERING IMPLICATION

A guarantee is meaningful only when its cost and graph assumptions are stated.

Breadth and depth trade memory for commitment

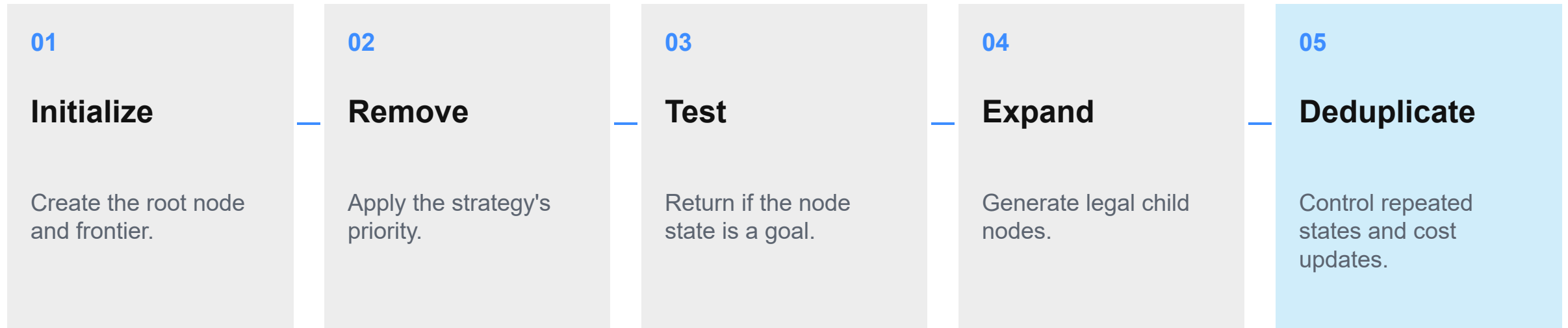
Breadth-first search

- FIFO frontier; expand nondecreasing depth.
- Complete for finite branching factor.
- Optimal when every step has equal cost.
- Time and space grow as $O(b^d)$.

Depth-first search

- LIFO frontier; follow one path deeply.
- Can fail in infinite-depth spaces.
- Not generally optimal.
- Space is $O(bm)$; time can be $O(b^m)$.

Generic graph search exposes the shared control loop



Correct duplicate handling is part of the algorithm, not a performance afterthought.

One search loop accepts a frontier policy

```
function SEARCH(problem, frontier):
    frontier.add(root(problem))
    best = {initial: 0}
    explored = empty set

    while not frontier.empty():
        node = frontier.remove()
        if goal(node.state):
            return path(node)
        if node.state in explored:
            continue
        explored.add(node.state)
        for child in expand(node):
            if child.g < best.get(child.state, infinity):
                best[child.state] = child.g
                frontier.add(child)
    return failure
```

Frontier choices

- BFS: FIFO by depth.
- DFS: LIFO.
- UCS: minimum g.
- Stable ties aid reproducibility.

Worked trace: BFS finds the shallow goal

SCENARIO

S connects to A and B; A connects to C; B connects to goal G; every edge costs 1.

- Start frontier: [S]. Remove S; add A, B.
- Remove A; explored is {S, A}; add C.
- Remove B; add G behind C.
- Remove C, then G; reconstruct S-B-G.

BFS returns depth 2 and cost 2; no depth-1 goal existed.

Completeness and optimality depend on assumptions

BFS

FIFO depth order

- Complete if b is finite.
- Optimal for equal step cost.
- High memory.
- Ignores cheaper deep paths.

DFS

LIFO depth order

- Low memory.
- May follow infinite path.
- Not optimal.
- Order strongly affects result.

UCS

Minimum g order

- Complete if costs $\geq \epsilon$.
- Optimal for positive costs.
- May expand many cheap paths.
- Needs cost updates.

Checkpoint: trace two frontier policies

CHECKPOINT

If S adds A then B, A adds G at depth 2, and B adds C then G at depth 3, which goal does BFS return and which path can DFS return?

01

Write the frontier after expanding S.

02

Apply FIFO and LIFO separately.

03

State the assumption about successor order.

Tree nodes and graph states are different objects

Node record

- Stores parent, action, depth, and g.
- Represents one path.
- Two nodes may share a state.
- Used for reconstruction.

State key

- Represents a world configuration.
- Needs stable equality and hashing.
- Drives explored membership.
- May map to a best known cost.

Why BFS and UCS can certify optimality

BFS: depth order + constant c ; UCS: remove n only when $g(n)$ is globally minimum

- With constant c , lower depth means lower path cost.
- BFS reaches no deeper goal before every shallower node.
- UCS frontier priorities are accumulated path costs.
- Positive costs prevent an unseen path from becoming cheaper after removal.

ENGINEERING IMPLICATION

The first goal removed is optimal under the stated ordering and cost conditions.

Cheaper rediscovery must update UCS

SCENARIO

S-A costs 5, S-B costs 1, and B-A costs 1; A leads to G with cost 1.

- S first discovers A with $g=5$ and B with $g=1$.
- UCS removes B before A.
- B rediscovers A with $g=2$, replacing the worse entry.
- A then reaches G with total cost 3.

Rejecting every duplicate at generation time would incorrectly preserve the cost-6 route.

Common implementation defects have distinct symptoms

Wrong ordering

Container mismatch

- BFS acts like DFS.
- UCS ignores g .
- Ties vary across runs.
- Guarantees no longer apply.

Wrong duplicate rule

State bookkeeping

- Cycles repeat.
- Cheaper paths are lost.
- Frontier grows unnecessarily.
- Parents become inconsistent.

Wrong contract

Problem interface

- Illegal actions expand.
- Costs can be nonpositive.
- Goal test accepts bad state.
- Failure is ambiguous.

Checkpoint: evaluate a guarantee

CHECKPOINT

An implementation uses BFS on edges with costs 1 and 10 and returns the first goal at minimum depth. Is the result guaranteed to have minimum cost?

01

Separate depth from path cost.

02

Construct a two-path counterexample.

03

Name the suitable replacement algorithm.

A shared interface makes guarantees testable

Keep domain expansion fixed and vary only the frontier policy when comparing search strategies.

Control

A fair comparison holds states, successors, costs, and goal behavior constant.

- Instrument nodes generated, expanded, maximum frontier, and solution cost.
- Use deterministic successor and tie order.
- Test trivial, cyclic, unreachable, cheaper-rediscovery, and scale cases.

Key takeaways

- 01 — **BFS, DFS, and UCS are one graph-search loop with different removal priorities.**
- 02 — Completeness and optimality always depend on explicit assumptions.
- 03 — Graph search needs immutable states and a correct duplicate-cost policy.
- 04 — Small traces reveal ordering and update defects before scale tests do.

Next: Informed search adds heuristics and develops Greedy Best-First Search and A-star.

Check your understanding

- Q1 Trace BFS and DFS on the same four-node graph.
- Q2 Give a graph where BFS is not minimum cost.
- Q3 Explain why UCS requires a positive lower bound on step cost.
- Q4 Derive the $O(b^d)$ BFS space term.
- Q5 Design a test that exposes a missing UCS cost update.